



**WYDZIAŁ FIZYKI
i INFORMATYKI STOSOWANEJ**
Uniwersytet Łódzki

Maciej Bujalski

Kierunek: informatyka

Specjalność: informatyka stosowana

Specjalizacja: aplikacje mobilne

Numer albumu: 386012

Rotacyjnie inwariantne sieci neuronowe

Praca magisterska

wykonana pod kierunkiem

dr Krzysztofa Podlaskiego

w Katedrze Systemów Inteligentnych, WFiIS UŁ

Łódź 2025

Spis treści

Wstęp	3
Cel i motywacja pracy	3
Opis pracy	4
Zakres tematyczny	5
Ujęte w zakresie	5
Poza zakresem	5
Artefakty pracy	6
Organizacja pracy	6
Podstawy teoretyczne	7
Wprowadzenie do sieci konwolucyjnych (CNN)	7
Operacja splotu	7
Receptywne pole	8
Nieliniowości i normalizacja	8
Pooling i głowa klasyfikacyjna	8
Regularizacja i trening	8
Triki architektoniczne (co faktycznie zmieniano)	8
Ekwawariancja translacyjna (kontrast do rotacyjnej)	9
Inwariancja translacyjna i rotacyjna	9
Problemy z rotacyjną inwariancją w klasycznych CNN	9
Przegląd literatury (E(2)-equivariant, CyCNN)	10
Opis zbiorów danych	10
MNIST (cyfry odręczne)	10
GTSRB Gray (znaki drogowe w odcienach szarości)	10
GTSRB RGB (znaki drogowe)	10
LEGO (obiekty 3d rzutowane na 2d)	10
Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów	10
Architektury modeli - stan faktyczny na podstawie kodu i szczegóły (VGG-E, ResNet-56, CyCNN)	10
VGG - wariant E (VGG-19, „3×3 everywhere”)	10
ResNet-56 (CIFAR, 6n+2 z n=9)	11
Wersje cykliczne: CyVGG-E i CyResNet-56	11
Uzgodnienia I/O i implementacja	12
Standardowe CNN	12
Rotacyjnie inwariantne sieci (CyResNet, CyVGG)	12
Transformacje polarne: linearpolar vs logpolar	12
Implementacja i środowisko eksperymentalne	12
Python, PyTorch	12
Struktura projektu	12
Automatyzacja: skrypty trenowania, testowania, ewaluacji	12
Obsługa GPU, Docker, WSL	12

Organizacja logów, modeli, confusion matrixów	12
Eksperymenty	13
Scenariusze trenowania/testowania (opis JSON)	13
Pomiar skuteczności (accuracy, macierze pomyłek)	13
Śledzenie metryk: średnia, mediana, odchylenie standardowe	13
Analiza skuteczności względem rotacji	13
Ranking modeli	13
Porównanie wyników	13
CyCNN vs klasyczne CNN	13
VGG vs CyVGG	13
Resnet vs CyResNet	13
CyVGG vs CyResNet	13
Wpływ transformacji (linearpolar vs logpolar)	13
Wydajność na różnych zbiorach	13
Wnioski	13
Skuteczność rotacyjnych architektur	13
Wnioski z automatyzacji i systematyzacji ewaluacji	13
Propozycje dalszych badań	13
Aneks	13
Listingi kodów	13
Dodatkowe wykresy, tablice wyników	13
Bibliografia	14

Wstęp

Obrazy otaczają nas z każdej strony: od zdjęć ze smartfonów, zdjęcia satelitarne, przez monitoring miejski, katalogi produktów i systemy kontroli jakości na liniach produkcyjnych, po systemy wspomagania jazdy. Choć współczesne modele rozpoznawania obrazu radzą sobie bardzo dobrze, w praktyce bywają wrażliwe na pozornie drobne zmiany takie jak obrócenie obiektu o kilkanaście stopni czy niewielki przechył kamery. To, co dla człowieka jest naturalne i natychmiast rozpoznawalne (znak drogowy pod kątem, cyfra obrócona na kartce), dla klasycznej konwolucyjnej sieci neuronowej bywa problemem. Rdzeń trudności to brak naturalnej inwariantności względem rotacji: standardowe CNN-y „z definicji” lepiej radzą sobie z przesunięciami niż z obrotami [1], [2].

W ostatnich latach pojawiło się kilka dróg domknięcia tej luki. Jedną to poszerzanie danych o zrotowane przykłady - poprawia odporność, ale wydłuża trening i nie gwarantuje uogólnienia na wszystkie kąty. Druga to architektury z wbudowaną geometrią: sieci grupowo równoważne (G-CNN, E(2)-equivariant) [3], [4], sieci cykliczne (CyCNN; w szczególności **CyVGG** i **CyResNet**) operujące na wielu orientacjach oraz przekształcenia do układów polarnych (linear-polar i log-polar), które „prostują” rotacje do przesunięć. Cel jest wspólny: by model rozpoznawał „to samo” niezależnie od orientacji, bez agresywnego dublowania danych.

Niniejsza praca skupia się na praktycznej weryfikacji tych podejść. Przygotowano zbiory obejmujące m.in. odręcznie napisane cyfry, znaki drogowe (w kolorze i w odcieniach szarości) oraz syntetyczne obiekty 3D rzutowane na 2D (klocki LEGO), a następnie rozszerzono je o kontrolowane rotacje. Zaimplementowano i porównano wybrane architektury rotacyjnie inwariantne i ich warianty bazowe w **PyTorchu** [5], mierząc wpływ transformacji (linear-polar vs. log-polar), wyboru architektury i zakresu kątów na jakość predykcji. Obliczenia realizowano na kartach: **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**, co skróciło czas trenowania i umożliwiło szeroki przegląd eksperymentów; środowisko uruchomieniowe ustandaryzowano z użyciem **Dockera** dla powtarzalności.

Celem pracy jest nie tylko pokazanie, że „da się” uzyskać odporność na rotacje, ale przede wszystkim wskazanie, **kiedy** i **jakim kosztem** ją osiągamy oraz które techniki przynoszą największy zysk względem klasycznych CNN-ów, jak wpływają na stabilność i szybkość uczenia, a także które konfiguracje są najpraktyczniejsze w realnych zastosowaniach (OCR, rozpoznawanie znaków, analiza obiektów technicznych). W dalszej części pracy przedstawiono podstawy, dane i augmentację, architektury, środowisko eksperymentalne, protokoły ewaluacji oraz wyniki z analizą i wnioskami.

Cel i motywacja pracy

Konwolucyjne sieci neuronowe (CNN) charakteryzują się zdolnością do analizy obrazów z zachowaniem niezmienniczości względem translacji. Jednak wciąż brakuje powszechnie uznanych architektur, które zapewniałyby inwariantność względem rotacji. Celem niniejszej pracy jest analiza skuteczności rozwiązań zaproponowanych w literaturze, ukierunkowanych na odporność modeli na rotację danych wejściowych (np. CyCNN [4]).

W ramach pracy zostały przygotowane wzbogacone zbiory danych, obejmujące m.in. zdjęcia odręcznie napisanych liter, znaki drogowe (zarówno w kolorze, jak i w odcieniach szarości) oraz obiekty 3D rzutowane na 2D (klocki LEGO), rozszerzone o kontrolowane rotacje obrazów.

Na podstawie tych zbiorów została przeprowadzona implementacja i ewaluacja wybranych architektur rotacyjnie inwariantnych z wykorzystaniem **PyTorch**. Obliczenia zostały wykonane przy użyciu kart graficznych **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**, co umożliwiło przyspieszenie trenowania. Otrzymane wyniki zostały porównane z rezultatami klasycznych sieci konwulucyjnych w celu oceny realnych korzyści z rozwiązań inwariantnych względem rotacji.

Opis pracy

Praca magisterska wykorzystuje zaawansowane technologie i narzędzia wspierające badania nad rotacyjnie inwariantnymi sieciami neuronowymi oraz ich zastosowaniem w przetwarzaniu obrazów. W realizacji projektu zastosowano następujące rozwiązania technologiczne:

- **Język programowania Python** - podstawowe narzędzie do implementacji algorytmów oraz obsługi frameworków uczenia maszynowego, dzięki wszechstronności i ekosystemowi bibliotek [6].

Frameworki uczenia maszynowego:

- **PyTorch** - elastyczny framework do szybkiego prototypowania i trenowania modeli [7].
- **TensorFlow** - narzędzie do budowy, trenowania i wdrażania modeli ML/DL [8].
- **Modele cykliczne (CyCNN)**. W pracy zostało przyjęte podejście, w którym obraz został przemapowany do współrzędnych (ρ, φ) . Dzięki temu obrót R_α staje się przesunięciem o α po osi φ . Konwolucje zostały zastąpione warstwami cylindrycznymi (CyConv) z cyklicznym paddingiem po φ . Dla każdego filtra zostało przygotowanych n orientacji; odpowiedzi zostały złożone z dodatkową osią „orientacja”. Obrót wejścia powoduje cykliczny shift po tej osi, a pooling po orientacjach daje inwariancję względem rotacji. Został ustawiony stały środek układu polarnych, stała siatka próbkowania oraz biliniarna interpolacja; padding po φ został ustawiony na cykliczny. Implementacja została wykonana w **PyTorchu** (punkt odniesienia: CyCNN [4]).
- **Wykorzystanie akceleracji GPU (NVIDIA)** - obliczenia zostały znacząco przyspieszone dzięki użyciu kart **RTX 3070 Ti 8 GB** oraz **RTX 3060 12 GB**. Frameworki wspierają **CUDA** oraz **cuDNN**, co umożliwia efektywne wykorzystanie zasobów GPU [9], [10].
- **Konteneryzacja za pomocą Dockera** - odizolowane środowiska uruchomieniowe ułatwiły replikację i współdzielenie projektu, także z obsługą GPU [11].

Zakres tematyczny

Niniejsza praca dotyczy odporności modeli klasyfikacji obrazów na rotacje w płaszczyźnie. Skupiono się na porównaniu klasycznych architektur z ich wersjami rotacyjnie inwariantnymi oraz na wpływie przekształceń polarnych na jakość predykcji. Badania zostały przeprowadzone na obrazach 2D i rotacjach planarnych. W szczególności zostały porównane warianty bazowe **VGG/ResNet** z wersjami cyklicznymi **CyVGG/CyResNet**, a także wpływ mapowania **linear-polar** i **log-polar**. Eksperymenty zostały wykonane na zbiorach **MNIST**, **GTSRB_gray**, **GTSRB_RGB** i **LEGO** z kontrolowanymi rotacjami.

Ujęte w zakresie

- **Architektury modeli:** zostały zaimplementowane i porównane warianty bazowe **VGG** oraz **ResNet** [12], [13], a także ich wersje cykliczne **CyVGG** i **CyResNet** (modele rotacyjnie inwariantne) [4].
- **Przekształcenia polarne:** została oceniona użyteczność mapowania **linear-polar** oraz **log-polar** jako etapów wstępnego przetwarzania służących „prostowaniu” rotacji do przesunięć [4], [14].
- **Zbiory danych:** zostały wykorzystane następujące zbiory: **MNIST** (odręczne cyfry, 28x28 → 32x32, grayscale) [15], **LEGO** (syntetyczne obiekty 3D rzutowane na 2D, 96x96, grayscale), **GTSRB** (znaki drogowe, 32x32, grayscale) oraz **GTSRB_RGB** (wersja kolorowa) [16]. Zbiory zostały rozszerzone o kontrolowane rotacje oraz zostały przygotowane spójne podziały zbiorów na train/val/test.
- **Augmentacja i protokół:** zostały zdefiniowane zakresy kątów, liczba powtórzeń i podział na zbiory train/val/test (uczący/ walidacyjny/testowy), z możliwością powtórzeń trenowań.
- **Środowisko i implementacja:** został wykorzystany **PyTorch** z akceleracją **CUDA/cuDNN** na kartach **NVIDIA GeForce RTX 3070 Ti 8 GB** oraz **NVIDIA GeForce RTX 3060 12 GB** [7], [9], [10]. Przygotowane zostały skrypty w Pythonie do trenowania, testowania i ewaluacji [6]. Środowisko uruchomieniowe zostało ustandaryzowane z użyciem **Dockera** [11].
- **Metryki i analiza:** została przeprowadzona ocena jakości (accuracy, macierze pomyłek), analiza stabilności (średnia/mediana/odchylenie standardowe), wpływ kąta rotacji na skuteczność oraz koszt obliczeniowy (czas trenowania, rozmiar modelu).

Poza zakresem

- Detekcja obiektów i segmentacja - w pracy rozpatrywana jest wyłącznie klasyfikacja [17], [18].
- Inwariancja względem skali, ścinania i pełnych przekształceń afinicznych - analizowana jest tylko rotacja w płaszczyźnie [19], [20].
- Rotacje w geometrii 3D oraz zagadnienia widzenia stereo - pozostają poza zakresem [21], [22].

- Trening na bardzo dużych korpusach z pre-treningiem self-supervised oraz szerokim AutoML/hyper-search - nie został realizowany [23], [24], [25].
- Odporność na silne zakłócenia (szum, okluzje) - poza zakresem; skupiono się wyłącznie na rotacji [26], [27].

Artefakty pracy

- Repozytoria z kodem, skryptami i plikami konfiguracyjnymi (PyTorch).
- Pliki konfiguracyjne eksperymentów i opis danych.
- Wytrenowane wagi modeli (wybrane checkpoints) oraz raporty z ewaluacji.
- Tekst pracy z dokumentacją eksperymentów i wnioskami.

Organizacja pracy

Struktura pracy została ułożona tak, by od podstaw przejść do wyników. W rozdziale **Podstawy teoretyczne** zostały zebrane pojęcia i narzędzia: CNN, inwariancja/ekwawariancja, przekształcenia polarne oraz prace pokrewne (G-CNN, E(2)-equivariant, CyCNN). W **Opisie zbiorów danych** zostały przedstawione MNIST, LEGO, **GTSRB_gray** i **GTSRB_RGB** oraz sposób augmentacji (rotacje, podziały train/val/test). W **Architekturach modeli** zostały opisane warianty bazowe **VGG/ResNet** oraz wersje cykliczne **CyVGG/CyResNet**, wraz z transformacjami linear-polar / log-polar. Rozdział **Implementacja i środowisko** zawiera szczegóły techniczne: **PyTorch**, **CUDA** i **cuDNN** (RTX 3070 Ti 8 GB, RTX 3060 12 GB), **Docker**, strukturę projektu i skrypty. W **Eksperymentach** zostały zdefiniowane scenariusze, metryki i sposób ewaluacji. Dalej, w **Porównaniu wyników**, zostały zestawione modele (VGG vs. CyVGG, ResNet vs. CyResNet, wpływ transformacji) i omówiona stabilność/czas. Na końcu **Wnioski** zbierają najważniejsze **obserwacje** i wskazują kierunki dalszych badań; **Aneks** zawiera kody i dodatkowe wykresy.

Podstawy teoretyczne

Wprowadzenie do sieci konwolucyjnych (CNN)

CNN zostały zaprojektowane pod dane siatkowe (obrazy 2D). Kluczowe cechy to **lokalne receptywne pola**, **współdzielenie wag** oraz **splot**, co daje skalowalność na duże obrazy i lepsze uogólnianie niż pełne połączenia. Zamiast patrzeć na cały kadr naraz, ten sam filtr „przesuwa się” po lokalnych oknach i uczy detektorów krawędzi, tekstur i kształtów, a wyżej - bardziej złożonych cech. [1], [28] Dla przesunięcia $\mathcal{T}_t$ i jądra K mamy ekwawariancję translacyjną:

$$\mathcal{T}_t(X) * K = \mathcal{T}_t(X * K).$$

Czyli: jak przesuniemy obraz, to mapa cech również przesuwa się o ten sam wektor. Inwariancję na przesunięcia „domyka się” w praktyce poolingiem (lokalnym/globalnym) albo stride’em - rzadsze próbkowanie sprawia, że decyzja nie zależy od dokładnej pozycji obiektu. [1], [2]

Operacja splotu

Intuicyjnie: ten sam mały filtr „przesuwa się” po obrazie i liczy sumę ważoną pikseli. Dzięki temu zostało osiągnięte współdzielenie wag (liczba parametrów nie zależy od H, W) oraz lokalność tych obliczeń [1], [2].

Kształty.

Wejście: $X \in \mathbb{R}^{C_{in} \times H \times W}$. Zestaw jąder: $K \in \mathbb{R}^{C_{out} \times C_{in} \times k \times k}$. Wyjście: $Y \in \mathbb{R}^{C_{out} \times H' \times W'}$.

Definicja (pojedynczy kanał wyjściowy c).

$$Y_c(u, v) = \sum_{i=1}^{C_{in}} \sum_{a,b} K_{c,i}(a, b) X_i(u - a, v - b).$$

W praktyce frameworki liczą **korelację krzyżową** (bez odwracania jądra), ale w API funkcja nazywa się conv [2]. Przyczym jest to bez znaczenia dla trenowania, bo sieć i tak nauczy się właściwych wag.

Stride, padding, rozmiary

Parametry „geometrii” warstwy:

- **padding** p - ile pikseli dopisujemy na brzegach,
- **stride** s - co ile pikseli przesuwamy okno,
- **dilacja** d - „rozciąga” jądro (dziurki między próbkami).

Rozmiar wyjścia (dla jądra $k \times k$):

$$H' = \left\lfloor \frac{H + 2p - d(k - 1) - 1}{s} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W + 2p - d(k - 1) - 1}{s} \right\rfloor + 1.$$

Typowe ustawienia.

- *valid*: $p = 0$ - mapy cech maleją.

- *same* (dla $s = 1$): $p = \lfloor k/2 \rfloor$ - wyjście ma $H' = H$, $W' = W$.
- *stride* > 1 : wbudowane „podsamplingowanie” (mniej obliczeń, mniejsza rozdzielczość).
- *dilacja* > 1 : większe **efektywne** pole widzenia bez nowych parametrów (użyteczne w detekcji/segmentacji) [2].

Receptywne pole

Receptywne pole to fragment obrazu, który „widzi” dana aktywacja. Teoretycznie rośnie ono z głębokością; dla kerneli k_ℓ i kroków s_ℓ :

$$R_1 = k_1, \quad R_\ell = R_{\ell-1} + (k_\ell - 1) \prod_{j < \ell} s_j.$$

W praktyce **efektywne** receptywne pole ma największy wpływ w centrum i słabnie ku brzegom (rozkład zbliżony do Gaussa) - dlatego w bazowych VGG/ResNet zostały dobrane głębokości tak, by objąć cały obiekt przy rozdzielczościach użytych w eksperymentach. W wersjach **CyCNN** po mapowaniu do (ρ, φ) i z **cyklicznym paddingiem** po φ sieć „widzi” pełny zakres orientacji bez artefaktów na brzegach [4], [29].

Nieliniowości i normalizacja

Blok konwolucyjny został utrzymany **identyczny we wszystkich wariantach** (bazowych i cyklicznych), bo celem porównania jest wpływ **rotacji**, a nie doboru aktywacji. Normalizacja została zastosowana standardowo, aby stabilizować uczenie; w wariantach **CyCNN** statystyki były liczone łącznie po osi **orientacji**, tak żeby **nie faworyzować żadnego kąta** [4], [30].

Pooling i głowa klasyfikacyjna

W modelach **CyCNN** inwariancja względem rotacji została uzyskana przez **pooling po orientacjach** (agregacja cech po osi kątów). Dalej, dla wszystkich modeli, zastosowany został **global average pooling** (GAP) i pojedyncza warstwa liniowa w klasyfikatorze. Głowa została celowo utrzymana taka sama w bazowych i cyklicznych wariantach, żeby izolować wpływ części „rotacyjnej” [4], [31].

Regularizacja i trening

Protokół trenowania został **zamrożony** między wariantami (te same: liczba epok, rozmiar batcha, budżet obliczeń, wczesne zatrzymanie lub brak - zgodnie z planem eksperymentu). Augmentacje ograniczono do tych **niezależnych od rotacji** w testach „czysto architektonicznych”; augmentacja rotacją była używana tylko w eksperymentach kontrolnych, żeby pokazać różnicę między „augmentacją” a „architekturą” (szczegóły w rozdziale z protokołem).

Triki architektoniczne (co faktycznie zmieniano)

- **Bazy:** VGG-19 (bloki 3×3) i ResNet-56 (wariant CIFAR, bloki 3×3).

- **Wersje cykliczne:** podmiana $\text{Conv} \rightarrow \text{CyConv}$, dodanie osi **orientacja** i **cykliczny padding** po φ ; pozostałe elementy (głębokość, liczba kanałów) zostały dobrane tak, by utrzymać *porównywalny budżet parametrów/FLOPs* względem baz.
- **Bez innych tricków:** nie wprowadzano zmian niezwiązanych z rotacją, żeby nie mieszać efektów [4].

Ekwawariancja translacyjna (kontrast do rotacyjnej)

Dla przesunięcia $\mathcal{T}_t$ i splotu zachodzi:

$$\mathcal{T}_t(X) * K = \mathcal{T}_t(X * K),$$

co tłumaczy, dlaczego klasyczne CNN dobrze radzą sobie z translacją [2]. Brak analogicznego mechanizmu dla rotacji motywuje przekształcenia polarne i/lub modele cykliczne dalej.

Inwariancja translacyjna i rotacyjna

Niech $\mathcal{T}_t$ oznacza przesunięcie o wektor t , $\mathcal{R}_\alpha$ obrót o kąt α , a Φ - mapę cech/model.

Ekwawariancja (przewidywalna zmiana reprezentacji):

$$\Phi(\mathcal{T}_t X) = \Pi_t \Phi(X), \quad \Phi(\mathcal{R}_\alpha X) = \Pi_\alpha \Phi(X),$$

gdzie Π to „działanie” grupy na cechach (dla translacji - przesunięcie map cech, dla rotacji - np. cykliczny shift po osi „orientacja”). [1], [32]

Inwariancja (wynik nie zależy od transformacji):

$$\Phi(\mathcal{T}_t X) = \Phi(X), \quad \Phi(\mathcal{R}_\alpha X) = \Phi(X).$$

W praktyce inwariancja jest uzyskiwana przez: - *translacja*: uśrednianie/próbkowanie rzadkie (GAP, stride),

- *rotacja*: (a) augmentację o obroty, (b) architekturę, która śledzi orientacje (oś „kąt”, potem pooling po orientacjach), (c) mapowanie do współrzędnych polarnych, gdzie obrót staje się przesunięciem po osi φ [2], [4], [14].

Problemy z rotacyjną inwariancją w klasycznych CNN

- **Kierunkowość filtrów.** Jeden kernel wykrywa wzorec głównie w jednej orientacji - bez dodatkowych mechanizmów sieć „gubi” obroty.
- **Augmentacja nie domyka całości.** Rotacje pomagają, ale wydłużają trening i zostawiają „dziury” między kątami (zwłaszcza przy małych krokach i ograniczonym budżecie).
- **Aliasing/interpolacja.** Obracanie dyskretnych obrazów wprowadza artefakty, co degradowuje sygnał [33].
- **Krawędzie i padding.** „same/zero” łamie symetrię przy brzegach - odpowiedzi nie są idealnie ekwawariantne.

- **Brak osi orientacji.** Standardowe CNN nie przechowują informacji „pod jakim kątem” aktywacja została wykryta - trudniej potem to „zwinąć” do wariantu niezależnego od kąta.

Przegląd literatury (E(2)-equivariant, CyCNN)

CyCNN (podejście użyte w pracy). Obraz został przemapowany do (ρ, φ) i przetwarzany warstwami cylindrycznymi (CyConv) z **cyklicznym paddingiem** po osi φ . Dla każdego filtra użyto n orientacji (grupa C_n). Obrót wejścia z C_n wywołuje **cykliczny shift** po osi orientacji (ekwawariancja), a **pooling po orientacjach** daje **inwariancję**. W badaniach zostały użyte CyVGG i CyResNet [4].

E(2)-equivariant / steerable CNNs (kontekst). Sploty grupowe i sterowalne jądra umożliwiają ekwawariancję względem translacji i rotacji (nawet dla ciągłych kątów) w grupie $E(2)$. Wymagają projektu jąder zgodnie z reprezentacjami grupy i zwykle większego kosztu obliczeń. Traktowane są tu jako tło teoretyczne [3], [34], [35].

Opis zbiorów danych

MNIST (cyfry odręczne)

GTSRB Gray (znaki drogowe w odcieniach szarości)

GTSRB RGB (znaki drogowe)

LEGO (obiekty 3d rzutowane na 2d)

Sposób augmentacji danych: zakresy rotacji, łączenie zbiorów

Architektury modeli - stan faktyczny na podstawie kodu i szczegóły (VGG-E, ResNet-56, CyCNN)

Przegląd. Użyte zostały bazy **VGG** (wariant **E**) i **ResNet** (wariant **56**) w ustawieniu „cifarowym” (obrazy 32×32): konwolucje 3×3 z padding=1, w VGG okresowy MaxPool2d(2), w ResNet zmniejszanie rozdzielczości przez stride=2. Na końcu **global average pooling** (GAP) i **klasyfikator** (w VGG: dwuwarstwowe połączenia $512 \rightarrow 512 \rightarrow C$ z dropoutem). W bazowym VGG został użyty wariant z **BatchNorm** (VGG_bn) [12], [13].

VGG - wariant E (VGG-19, „3×3 everywhere”)

Układ wg [12], dostosowany do 32×32 (CIFAR):

- **Blok 1:** 64, 64 $\rightarrow$ MaxPool (32 $\rightarrow$ 16)
- **Blok 2:** 128, 128 $\rightarrow$ MaxPool (16 $\rightarrow$ 8)

- **Blok 3:** 256×4 konwolucje $\rightarrow$ MaxPool ($8 \rightarrow 4$)
- **Blok 4:** 512×4 konwolucje $\rightarrow$ MaxPool ($4 \rightarrow 2$)
- **Blok 5:** 512×4 konwolucje $\rightarrow$ MaxPool ($2 \rightarrow 1$)

Po części konwolucyjnej: $\text{AdaptiveAvgPool}(1, 1) \rightarrow$ **klasyfikator** $512 \rightarrow 512 \rightarrow C$ (dropout). W bazie: VGG_bn.

ResNet-56 (CIFAR, $6n+2$ z $n=9$)

Schemat jak w [13] dla CIFAR:

- start: Conv 3×3 , dalej **3 grupy** po $n=9$ bloków BasicBlock,
- **Grupa 1 (16 kanałów):** stride 1, 9 bloków,
- **Grupa 2 (32 kanały):** pierwszy blok stride 2 (zmniejszenie rozdż.),
- **Grupa 3 (64 kanały):** pierwszy blok stride 2.

BasicBlock: Conv $3 \times 3 \rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Conv $3 \times 3 \rightarrow$ BN + skrót.

Downsample: **opcja A** (CIFAR) - podpróbkowanie i dopełnienie kanałów.

Zakończenie: GAP $\rightarrow$ **warstwa liniowa** $64 \rightarrow C$.

Wersje cykliczne: CyVGG-E i CyResNet-56

- **Conv $\rightarrow$ CyConv.** Każdą Conv2d zastąpiono CyConv2d (warstwa cylindryczna; interfejs jak Conv2d: 3×3 , padding=1, obsługa stride/dilacji).
- **Oś orientacji.** Odpowiedzi składają się z dodatkową osią *orientacja* (liczba orientacji n z konfiguracji). Obrót wejścia odpowiada cyklicznemu przesunięciu po tej osi (**ekwawariancja**).
- **Cykliczny padding po φ .** „Zawijanie” po osi kątowej usuwa artefakty brzegowe po mapowaniu do (ρ, φ) [4].
- **Inwariancja.** Przed klasyfikatorem zastosowano **agregację po orientacjach** (średnia/maksimum), co uniezależnia wynik od kąta.
- **Reszta bez zmian.** Układ bloków, liczby kanałów i **klasyfikator** (GAP/Linear lub GAP + połączenia $512 \rightarrow 512 \rightarrow C$) zostały zachowane jak w bazach, aby porównanie było uczciwe (zbliżony budżet parametrów/FLOPs) [4].

Uzgodnienia I/O i implementacja

- **Kanały wejścia.** 1 kanał dla MNIST, GTSRB_gray, LEGO (w kodzie GTSRB-custom); 3 kanały dla GTSRB_RGB.
- **Liczba klas.** MNIST 10, GTSRB 43 (gray i GRB), LEGO 50 (zgodnie z `get_num_classes`).
- **Implementacja.** CyConv2d korzysta z dedykowanego kernela CUDA (CyConv2d_cuda) i bufora roboczego na GPU (przyspieszenie obliczeń, m.in. wariant Winograda). Część „rotacyjna” jest enkapsulowana w CyConv2d - kod modeli pozostaje w tym samym API [4].

Standardowe CNN

Jako bazy zostały użyte **VGG** (wariant **E** / **VGG-19**) i **ResNet-56** w układzie „cifarowym” (obrazy 32×32 oraz 96×97). Konwolucje 3×3 z `padding=1`, w VGG okresowy `MaxPool2d(2)`, w ResNet zmniejszanie rozdzielczości przez `stride=2`. Po części konwolucyjnej zastosowano **global average pooling (GAP)** i **klasyfikator** (w VGG: dwie warstwy w pełni połączone $512 \rightarrow 512 \rightarrow C$ z dropoutem). W wariantcie VGG został użyty model z **BatchNorm** (VGG_bn).

Modele te są z natury **ekwawariantne translacyjnie** (dobrze znoszą przesunięcia), ale nie posiadają wbudowanej inwariancji względem rotacji - to jest punkt odniesienia dla wersji cyklicznych [12], [13].

Rotacyjnie inwariantne sieci (CyResNet, CyVGG)

Wersje cykliczne (CyVGG-E, CyResNet-56) powstały przez **podmianę każdej Conv2d na CyConv2d** (warstwa cylindryczna). Przetwarzanie odbywa się w układzie (ρ, φ) ; po osi φ został użyty **padding cykliczny**, aby „zawinąć” kąty. Dla każdej warstwy została dodana **oś orientacji** (liczba orientacji n ustawiana w konfiguracji). Obrót wejścia o kąt z dyskretnego zbioru C_n przekłada się na **cykliczne przesunięcie** po tej osi (**ekwawariancja**), a **agregacja po orientacjach** (avg/max) przed klasyfikatorem daje **inwariancję** względem rotacji.

Topologia bloków, liczby kanałów i część klasyfikacyjna zostały utrzymane jak w bazach, żeby porównanie było **uczciwe** (zbliżony budżet parametrów/FLOPs). Implementacja wykorzystuje dedykowany kernel CUDA dla CyConv2d, co skraca czas obliczeń. Punktem odniesienia jest koncepcja **CyCNN** (mapowanie polarne + warstwy cylindryczne) [4].

Transformacje polarne: linearpolar vs logpolar

Implementacja i środowisko eksperymentalne

Python, PyTorch

Struktura projektu

Automatyzacja: skrypty trenowania, testowania, ewaluacji

Obsługa GPU, Docker, WSL

Organizacja logów, modeli, confusion matrixów

Eksperymenty

Scenariusze trenowania/testowania (opis JSON)

Pomiar skuteczności (accuracy, macierze pomyłek)

Śledzenie metryk: średnia, mediana, odchylenie standardowe

Analiza skuteczności względem rotacji

Ranking modeli

Porównanie wyników

CyCNN vs klasyczne CNN

VGG vs CyVGG

Resnet vs CyResNet

CyVGG vs CyResNet

[...] cyresnet56 uczył się dłużej niż cyvgg19, ale dawał bardziej stabilne wyniki, w szczególności w przypadku funkcji aktywacji typu logpolar. I jak to się mówi - nie można zjeść ciastka i mieć go też (ang. „You can’t eat your cake and have it too” [36]).

Wpływ transformacji (linearpolar vs logpolar)

Wydajność na różnych zbiorach

Wnioski

Skuteczność rotacyjnych architektur

Wnioski z automatyzacji i systematyzacji ewaluacji

Propozycje dalszych badań

Aneks

Listingi kodów

Dodatkowe wykresy, tablice wyników

Bibliografia

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: <https://www.deeplearningbook.org/>
- [2] V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning.” 2016. Available: <https://arxiv.org/abs/1603.07285>
- [3] T. Cohen and M. Welling, “Group equivariant convolutional networks,” in *Proceedings of the 33rd international conference on machine learning (ICML)*, 2016. Available: <https://proceedings.mlr.press/v48/cohen16.html>
- [4] J. Kim, W. Jung, H. Kim, and J. Lee, “CyCNN: A rotation invariant CNN using polar mapping and cylindrical convolution layers,” *arXiv preprint arXiv:2007.10588*, 2020, Available: <https://arxiv.org/abs/2007.10588>
- [5] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in neural information processing systems 32 (NeurIPS)*, 2019. Available: <https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [6] Python Software Foundation, “Python 3 documentation.” 2025. Available: <https://docs.python.org/3/>
- [7] PyTorch Core Team, “PyTorch documentation (stable).” 2025. Available: <https://pytorch.org/docs/stable/>
- [8] TensorFlow Authors, “TensorFlow API documentation.” 2025. Available: https://www.tensorflow.org/api_docs
- [9] NVIDIA Corporation, “CUDA toolkit documentation.” 2025. Available: <https://docs.nvidia.com/cuda/>
- [10] NVIDIA Corporation, “NVIDIA cuDNN developer guide.” 2025. Available: <https://docs.nvidia.com/deeplearning/cudnn/developer-guide/index.html>
- [11] Docker, Inc., “Docker documentation.” 2025. Available: <https://docs.docker.com/>
- [12] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014, Available: <https://arxiv.org/abs/1409.1556>
- [13] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2016, pp. 770–778. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html
- [14] B. S. Reddy and B. N. Chatterji, “An FFT-based technique for translation, rotation and scale-invariant image registration,” *IEEE Transactions on Image Processing*, vol. 5, no. 8, pp. 1266–1271, 1996, doi: 10.1109/83.506761.
- [15] Y. LeCun, C. Cortes, and C. J. C. Burges, “The MNIST database of handwritten digits.” 1998. Available: <http://yann.lecun.com/exdb/mnist/>
- [16] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, “The german traffic sign recognition benchmark: A multi-class classification competition,” in *Proceedings of the international joint conference on neural networks (IJCNN)*, 2011, pp. 1453–1460. doi: 10.1109/IJCNN.2011.6033395.

- [17] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask r-CNN,” in *Proceedings of the IEEE international conference on computer vision (ICCV)*, 2017. Available: https://openaccess.thecvf.com/content_ICCV_2017/papers/He_Mask_R-CNN_ICCV_2017_paper.pdf
- [18] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *MICCAI*, in LNCS, vol. 9351. 2015, pp. 234–241. Available: https://link.springer.com/chapter/10.1007/978-3-319-24574-4_28
- [19] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, Available: <https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf>
- [20] M. Jaderberg, K. Simonyan, A. Zisserman, and K. Kavukcuoglu, “Spatial transformer networks,” in *Advances in neural information processing systems (NIPS)*, 2015. Available: <https://proceedings.neurips.cc/paper/2015/file/33ceb07bf4eeb3da587e268d663aba1a-Paper.pdf>
- [21] C. Esteves, C. Allen-Blanchette, A. Makadia, and K. Daniilidis, “Learning SO(3)-equivariant representations with spherical CNNs,” in *Proceedings of the european conference on computer vision (ECCV)*, 2018, pp. 52–68. Available: https://openaccess.thecvf.com/content_ECCV_2018/html/Carlos_Esteves_Learning_SO3_Equivariant_ECCV_2018_paper.html
- [22] R. Hartley and A. Zisserman, *Multiple view geometry in computer vision*, 2nd ed. Cambridge University Press, 2004.
- [23] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *Proceedings of the 37th international conference on machine learning (ICML)*, PMLR, 2020. Available: <https://proceedings.mlr.press/v119/chen20j/chen20j.pdf>
- [24] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2020. Available: https://openaccess.thecvf.com/content_CVPR_2020/papers/He_Momentum_Contrast_for_Unsupervised_Visual_Representation_Learning_CVPR_2020_paper.pdf
- [25] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: Bandit-based configuration evaluation for hyperparameter optimization,” *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018, Available: <https://jmlr.org/papers/volume18/16-558/16-558.pdf>
- [26] D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” in *International conference on learning representations (ICLR)*, 2019. Available: <https://web.engr.oregonstate.edu/~tgd/publications/hendrycks-dietterich-benchmarking-neural-network-robustness-to-common-corruptions-and-perturbations-iclr2019.pdf>
- [27] T. DeVries and G. W. Taylor, “Improved regularization of convolutional neural networks with cutout.” 2017. Available: <https://arxiv.org/abs/1708.04552>
- [28] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, 1998.

- [29] W. Luo, Y. Li, R. Urtasun, and R. Zemel, “Understanding the effective receptive field in deep convolutional neural networks,” in *Advances in neural information processing systems (NeurIPS)*, 2016.
- [30] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *International conference on machine learning (ICML)*, 2015.
- [31] M. Lin, Q. Chen, and S. Yan, “Network in network,” in *International conference on learning representations (ICLR)*, 2014.
- [32] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković, “Geometric deep learning: Grids, groups, graphs, geodesics, and gauges,” *arXiv preprint arXiv:2104.13478*, 2021, Available: <https://arxiv.org/abs/2104.13478>
- [33] A. Azulay and Y. Weiss, “Why do deep convolutional networks generalize so poorly to small image transformations?” in *Journal of vision*, 2019, pp. 1–14.
- [34] M. Weiler and G. Cesa, “General $e(2)$ -equivariant steerable CNNs,” *arXiv preprint arXiv:1911.08251*, 2019, Available: <https://arxiv.org/abs/1911.08251>
- [35] T. S. Cohen, M. Geiger, and M. Weiler, “A general theory of equivariant CNNs on homogeneous spaces,” in *Advances in neural information processing systems (NeurIPS)*, 2019. Available: <https://papers.neurips.cc/paper/9114-a-general-theory-of-equivariant-cnns-on-homogeneous-spaces.pdf>
- [36] T. J. Kaczynski, “Industrial society and its future,” *The Washington Post*, Sep. 1995, Available: <https://www.washingtonpost.com/wp-srv/national/longterm/unabomber/manifesto.text.htm>